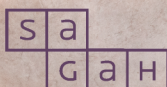


PROCESSAMENTO DE LINGUAGEM NATURAL

Juliano Vieira Martins



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Classificação de textos — Utilizando Python para construir e treinar modelos de *machine learning*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever processos de manipulação de dados para utilizar classificadores de texto.
- Analisar o classificador Naïve Bayes.
- Aplicar o classificador Naïve Bayes em um estudo prático.

Introdução

A classificação de texto compreende o processo de atribuir rótulos ou etiquetas de categorias ao texto de acordo com seu conteúdo, uma das tarefas fundamentais do processamento de linguagem natural (PLN), que tem aplicações amplas, como análise de sentimentos, rotulagem de tópicos, detecção de *spam*, de intenção e de idioma, entre outras. São vários os contextos em que se pode usar a classificação de texto, por exemplo, classificação de textos curtos, como *tweets*, de chamadas para notícias e de mensagens SMS, além da classificação de texto na organização de documentos quando se deseja realizar análises de clientes, compreender artigos de mídia ou, ainda, extrair contextos de contratos legais.

Dados em forma de texto estão em toda parte, como em *e-mails*, *chats*, páginas da *web*, mídias sociais, *tickets* de suporte, respostas a pesquisas, etc. Esses dados podem ser fontes de informação extremamente relevantes, no entanto extrair informações destas pode ser trabalhoso e demorado em virtude de sua natureza não estruturada. Por isso, as empresas têm recorrido à classificação de texto para estruturar textos

de maneira rápida e econômica, com o objetivo de aprimorar a tomada de decisões e automatizar os processos.

Neste capítulo, você compreenderá o processo de preparação dos dados antes de serem submetidos à classificadores de texto, além de aprender como o algoritmo de classificação Naïve Bayes é construído e verificar suas características. Por fim, o algoritmo Naïve Bayes será aplicado em um estudo prático de classificação de texto utilizando a linguagem de programação Python.

1 Obtenção e preparação dos dados

Um classificador de texto tem pouca utilidade se os dados necessários para o treinamento, que tem por objetivo capacitá-lo, não forem precisos. Assim como os humanos conseguem fazer previsões com base em fatos já vistos ou vividos, os algoritmos de aprendizagem de máquina podem realizar previsões, comumente chamadas de “previsões”, aprendendo com exemplos anteriores. Assim, um classificador que consegue prever com altos níveis de precisão depende inteiramente da obtenção dos dados de treinamento corretos, o que significa reunir amostras de dados que melhor representam os resultados que se deseja prever. Por exemplo, quando se tem a necessidade de prever a intenção das conversas em uma sala de bate-papo, é preciso identificar e reunir previamente as conversas de sala de bate-papo que representam as diferentes intenções que se deseja prever. Com base nisso, se o modelo de previsão do classificador for treinado com dados poucos representativos para o objetivo, o classificador fornecerá resultados ruins (JURAFSKY; MARTIN, 2019).

Obtenção dos dados

Quando tratamos sobre classificação de texto, podemos obter os dados das mais variadas fontes, sejam próprias, sejam de terceiros. A pergunta a ser feita nesse momento é “para qual objetivo de classificação são os dados?”. A partir da resposta, tem-se a orientação de em qual local se deve buscar obter os dados. Por exemplo, se o objetivo é classificar sentimentos de clientes, podemos usar dados internos gerados a partir dos aplicativos e das ferramentas usados diariamente, como sistemas de atendimento ao cliente, formulários de

satisfação do cliente, etc., que, geralmente, oferecem a opção para exportar os dados para arquivos de formato *comma-separated values* (CSV — valores separados por vírgula) ou, ainda, *tab-separated values* (TSV — valores separados por tabulação).

Além das fontes internas para obtenção dos dados, outra opção consiste em usar fontes externas disponíveis na *web*. Assim, os dados podem ser coletados a partir do emprego de alguma ferramenta de “coleta de dados *web*” ou da *application programming interface* (API) — interface de programação de aplicativo (ou aplicação) —, um *software* intermediário que possibilita a comunicação entre dois aplicativos, ou, ainda, de algum conjunto de dados públicos (*public dataset*). A seguir, listamos alguns conjuntos de dados disponíveis ao público.

- Reuters news dataset (classificação de tópicos): com cerca de 20 mil artigos.
- 20 Newsgroups (classificação de tópicos): com aproximadamente 20 mil documentos de grupos de notícias.
- Amazon Product Reviews (análise de sentimentos): com cerca de 143 milhões de avaliações.
- IMDB reviews (análise de sentimentos): com cerca de 25 mil críticas de filmes.
- Twitter Airline Sentiment (análise de sentimentos): com aproximadamente 15 mil *tweets* sobre companhias aéreas.
- Spambase (setecção de *spam*): com cerca de 4,6 mil *e-mails* rotulados.
- SMS Spam Collection (detecção de *spam*): com aproximadamente 5,5 mil mensagens SMS rotuladas.



Saiba mais

Busque na internet cada nome dos conjuntos de dados públicos listados anteriormente, por exemplo, busque Reuters news dataset e procure o *link* oficial do *dataset*. Lá, são apresentadas as características de cada conjunto de dados, como a quantidade de amostras, a maneira como estão rotuladas, etc. Assim, será possível conhecer um pouco mais sobre cada um deles e compreender seus respectivos objetivos.

Para que o leitor possa compreender na prática como obter, explorar e preparar os dados, este capítulo fornecerá um passo a passo em código Python (PYTHON, c2020) de como fazê-lo. Para isso, é necessário utilizar algumas ferramentas e bibliotecas disponíveis, conforme explicado a seguir.

Google Colab

Também conhecido como Colaboratory, trata-se de um ambiente Jupyter Notebook gratuito que não requer configuração, além de ser executado inteiramente na nuvem de computadores (GOOGLE COLABORATORY, c2020). Na plataforma do Colaboratory (Figura 1), é possível escrever e executar códigos, salvar e compartilhar análises e acessar recursos de computação diretamente pelo navegador. O Jupyter Notebook é um ambiente de desenvolvimento interativo totalmente baseado na *web*, no qual podemos escrever blocos de código, blocos de textos explicativos, equações e visualizar gráficos ou os resultados da execução de códigos. Para acessar o Google Colab, é necessário ter uma conta no Gmail.

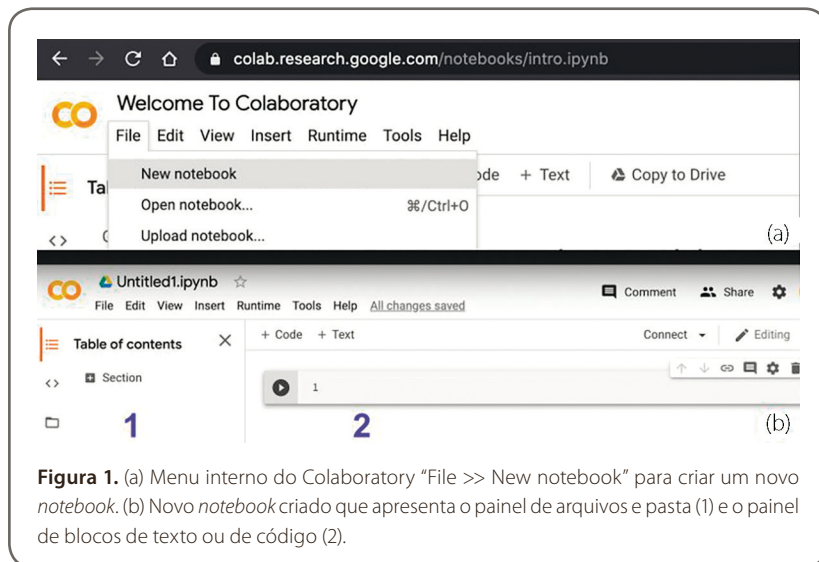


Figura 1. (a) Menu interno do Colaboratory “File >> New notebook” para criar um novo *notebook*. (b) Novo *notebook* criado que apresenta o painel de arquivos e pasta (1) e o painel de blocos de texto ou de código (2).

Agora que você foi introduzido ao Colaboratory, a cada exemplo de código, o convidamos a criar um novo arquivo no Colaboratory (Figura 1a) e, então, a criar um novo bloco de código (Figura 1b) na plataforma e executá-lo. Dessa forma, será possível observar na prática os resultados das teorias abordadas.

SMS Spam Collection Data Set

Atualmente, o repositório de aprendizagem de máquina UC Irvine Machine Learning Repository (UCI) (DUA; GRAFF, 2019) mantém mais de 490 conjuntos de dados como um serviço para a comunidade de aprendizagem de máquina. É possível visualizar todos os conjuntos de dados por meio da interface pesquisável no *site*. O conjunto de dados SMS Spam Collection Data Set é um dos conjuntos disponibilizados pelo UCI.

Como um primeiro passo, o leitor pode criar um novo *notebook* no Colaboratory (Figura 1b), adicionar um novo bloco de código e testar a versão do Python disponível no ambiente com o seguinte comando:

```
!python --version
```

Veja que, em vez de “código”, o exemplo foi chamado de “comando”, já que no código apresentado existe um “ponto de exclamação” no início, o qual, por sua vez, indica que o código deve ser executado como se fosse em um ambiente de linha comando de um terminal, ou seja, ter o mesmo comportamento de um comando executado em uma *command line interface* (CLI). Assim, para executar comandos Linux nas caixas de código basta indicar com um ponto de exclamação no início do comando. O resultado do comando do exemplo consiste em exibir a versão do Python.

Exploração dos dados

Agora que o leitor já foi introduzido aos conjuntos de dados públicos e ao Colaboratory, o próximo passo será obter o *dataset* de mensagens já rotuladas como *spam/ham*. A seguir, apresentamos o código para realizar o *download* do *dataset*. Para isso, são utilizadas três bibliotecas, comumente chamadas de módulos ou pacotes.



Exemplo

```
import requests # Biblioteca HTTP
import zipfile # Biblioteca de compressão de arquivos
import io # Biblioteca para lidar com Entrada e saídas.

# URL do arquivo TSV
url = 'https://archive.ics.uci.edu/ml/machine-learning-
-databases/00228/smsspamcollection.zip'

# Download do arquivo TSV
r = requests.get(url)

# Instanciando o compressor ZipFile
z = zipfile.ZipFile(io.BytesIO(r.content))

# Extração (descompressão) dos dados
z.extractall()
```

Ao executar o bloco de código exemplificado, uma cópia do *dataset SMSSpamCollection* será baixada para dentro da pasta `/content`, a pasta principal do painel de arquivos e das pastas do Colaboratory (Figura 1 b1). Assim, o *dataset* baixado estará no endereço `/content/SMSSpamCollection`. De posse dos dados, é possível explorar o conteúdo do conjunto e verificar o que de fato o *dataset* contém. Para isso, será utilizada a biblioteca Pandas, extremamente eficiente para esse trabalho (PANDAS, 2020). O Pandas Data-Frame é uma estrutura de dados tabular, bidimensional, mutável em tamanho e que pode ser heterogênea, com eixos rotulados (linhas e colunas). Assim, um *data frame* é composto por três componentes principais: os dados, as linhas e as colunas.



Exemplo

```
import pandas as pd

# Carregar o dataset de mensagens para um DataFrame
df = pd.read_table('/content/SMSSpamCollection',
header=None, encoding='utf-8')

# Imprimir informações úteis sobre o conjunto de dados
(opcional)
print(df.info())

# Imprimir as 5 primeiras linhas do conjunto de dados
(opcional)
print(df.head())

# Variável para as classes para evitar ficar usando a
notação df[0]
classes = df[0]

# Imprimir a distribuição das classes (opcional)
print(classes.value_counts())
```

Ao executar o bloco de código descrito, serão exibidas as informações sobre os dados (método `info()` do `DataFrame`) e as primeiras linhas da tabela (método `head()` do *data frame*). Uma das informações importantes consiste na quantidade de registros (instâncias ou amostras) de que o *dataset* dispõe. No caso do exemplo do *dataset* apresentado, existem 5.572 registros, começando com o identificador (id) 0 até o 5.571. Além disso, as primeiras 5 linhas apresentadas pelo método `head()` são os dados reais existentes no conjunto: assim, na coluna 0 (à esquerda), estão dispostos os rótulos das classes, e, na coluna 1 (à direita), as mensagens. Ainda, a última linha de código apresentará como as classes (rótulos) estão distribuídas para os registros. No caso do *dataset* em questão, a distribuição das classes é de 4.825 para *ham* e 747 para *spam*.

Preparação dos dados

Até agora, discutimos sobre como obter o *dataset* e verificar seu conteúdo, contudo ainda faltam alguns passos importantes a realizarmos para que o *dataset* esteja pronto para uso em um classificador. Um desses passos é o pré-processamento do texto que o *dataset* contém, que representa uma das tarefas mais importantes no PLN. Por exemplo, convém remover todos os sinais de pontuação dos documentos de texto antes que eles possam ser usados para classificação de texto, e, do mesmo modo, extrair os números das sequências de texto. Escrever códigos manualmente para essas tarefas de pré-processamento exige muito esforço, além da possibilidade de ocorrência de falhas. Para facilitar a tarefa de pré-processamento, tendo em vista a sua importância, existem as expressões regulares, comumente denominadas *regex* (JURAFSKY; MARTIN, 2019).

Uma expressão regular compreende uma sequência de caracteres que descreve um “padrão” de busca em texto que pode ser usado para encontrar ou substituir as expressões em uma *string*. Para melhor compreendê-la, apresentamos um exemplo de código para identificar as palavras compostas por algum caractere especial, ou seja, os números, *e-mails*, telefones, URL, etc. Para implementar expressões regulares, pode-se usar o pacote *re* (*regular expression*) do Python.



Exemplo

```
# Biblioteca de expressão Regulares
import re

# Padrão de expressão regular para encontrar números e
# caracteres especiais
pattern = re.compile(r'[\d@_!#$%^&*()<>?/\|}{~:~:]')

# Selecionar a coluna de mensagens ()
text_messages = df[1]

# Selecionar as 10 primeiras linhas
messages = text_messages[0:10]
```

```
# Para cada linha na lista de mensagens
for line in messages:
    # Criar lista de palavras dividindo a frase pelos
    # espaços em branco
    words = line.split(' ')
    # Para cada palavra na lista de palavras:
    for word in words:
        # Se algum caractere do padrão existir na palavra
        if re.search(pattern, word):
            # Imprime a palavra
            print(word)
```

Após a execução desse código, são exibidas palavras que contêm caracteres especiais, além de números, *e-mails*, telefones, URL. Vale ressaltar que o código do exemplo foi apenas informativo, já que nada se alterou no conjunto de dados.



Saiba mais

Para testar, de modo *online*, alguns padrões para expressões regulares, é possível utilizar algumas ferramentas disponíveis na *web*, a partir de uma busca na internet pela frase *regular expression online tester*.

O próximo passo para poder treinar um modelo de predição em um classificador de texto será a extração de recursos (*features*) do texto (JURAFSKY; MARTIN, 2019). Para isso, pode-se utilizar um método para transformar o texto em uma representação numérica, na forma de vetor, como ao criar um conjunto de todas as palavras presentes no texto e, então, realizar a contagem de frequência dessas palavras. A partir disso, deve-se selecionar nesse conjunto as palavras mais frequentes, que constituirão a base para a montagem dos vetores de *features*. Assim, em vez de submeter um texto ao classificador, por exemplo, uma mensagem de SMS, deve-se submeter um vetor de valores que representam as palavras que aparecem no texto de cada mensagem. Partindo do princípio de vetorizar o texto das mensagens, passa a fazer sentido remover ou substituir aquilo que não representa muito bem uma palavra, como os números, os endereços de *e-mails*, as URL, as pontuações, etc. Para isso, as expressões

regulares podem ser usadas novamente. Dessa vez, cada item será substituído de uma única vez em todas as linhas da coluna 1 do *data frame*, que contém as mensagens. Mas, antes disso, todas as mensagens serão transformadas para letras minúsculas, para manter certo padrão — por exemplo, devemos garantir que Love e love tenham a mesma importância.



Exemplo

```
# Alterar todas as palavras para minúsculas
processed_lines = text_messages.str.lower()

# Lista de tuplas com os padrões e novas palavras
patterns = [
    # Substituir endereços de email por 'emailaddress'
    (r'^.+@[^\.]*.\\.[a-z]{2,}$', 'emailaddress'),
    # Substituir URLs por 'webaddress'
    (r'^http://[a-zA-Z0-9\\-\\.]+\\.[a-zA-Z]{2,3}(\\/\\S*)?$$',
     'webaddress'),
    # Substituir símbolos monetários (Dólar e Euro) por
    'moneysymb'
    (r'£|\\$', 'moneysymb'),
    # Substituir números de telefone por 'phonenumbr'
    (r'^\\(?:\\d{3}\\)?\\s-?\\d{3}\\s-?\\d{4}$', 'phonenumbr'),
    # Substituir números por 'numbr'
    (r'\\d+(\\.\\d+)?', 'numbr'),
    # Remover pontuação (! ?)
    (r'[\\^w\\d\\s]', ' '),
    # Substitua dois ou mais espaços em branco por um
    único espaço
    (r'\\s+', ' '),
    # Remova os espaços em branco à esquerda e à direita
    (r'^\\s+|\\s+?$', '')
]

# Substituir no texto os padrões encontrados
for pattern, newword in patterns:
    processed_lines = processed_lines.str.replace(pattern,
newword)
```


No exemplo, existem duas peculiaridades interessantes que precisamos explicar para que você entenda o que está ocorrendo no código. A primeira reside no fato de que a variável `text_messages`, que representa a lista de mensagens da coluna 1 do *data frame*, é do tipo `pandas.core.series.Series`, e implementa uma instância da classe `str (string)` do Python. Consequentemente, a classe `str` tem o método `replace()`, que faz a substituição por texto literal ou expressão regular. A segunda peculiaridade está nas palavras escolhidas para a substituição, visto que nenhuma delas pode colidir com palavras já existentes no texto. Por exemplo, substituir um endereço de *e-mail* pela palavra *mail* fará com que a frequência da palavra *mail*, que pode existir no texto, aumente artificialmente. Assim, para realizar as substituições, deve-se sempre escolher palavras que não existem em um texto normal, como *emailaddress*, uma palavra segura para substituir um endereço de *e-mail*, já que dificilmente ela existe, organicamente, em algum texto de mensagem de SMS. No entanto, ainda que venha a existir, terá pouca ou nenhuma influência no resultado final.

Ainda, há dois passos que devem ser realizados antes que os textos das mensagens prossigam para a vetorização: o primeiro consiste em remover as palavras consideradas *stop words*, ou seja, uma “palavra de parada” (JURAFSKY; MARTIN, 2019), que tem valor irrelevante em meio a um conjunto de palavras, pois representa as palavras mais comuns de um idioma. Por exemplo, algumas *stop words* em inglês seriam *i, me, my, myself, we, our, ours, etc.* Geralmente, os motores de busca de páginas da *web* filtram essas palavras antes de iniciarem o processo de busca. Já o segundo passo refere-se a *stemmizar* algumas palavras que podem ser substituídas por uma variante morfológica, ou seja, trocar uma palavra por outra palavra de base equivalente (JURAFSKY; MARTIN, 2019). Por exemplo, substituir as palavras, em inglês, *programs, programmer, programing e programmers* por uma única palavra *program*, a palavra-base para aquelas. Para isso, podemos utilizar a biblioteca Natural Language Tool Kit (NLTK), ou *kit* de ferramentas de linguagem natural (NATURAL LANGUAGE TOOLKIT, 2020).



Exemplo

```
# Biblioteca de processamento de linguagem natural
import nltk
# arquivo de suporte ao stopwords
nltk.download('stopwords')
# arquivo de suporte ao stopwords
nltk.download('punkt')
# Biblioteca para gerar lista de stopwords
from nltk.corpus import stopwords
# Biblioteca para stemmizar palavras
from nltk.stem import PorterStemmer
# Biblioteca para tokenizar texto
from nltk.tokenize import word_tokenize

# Criar uma lista das stop words
stop_words = stopwords.words('english')

# Inicializar uma instância de Porter Stemmer
ps = PorterStemmer()

# para cada índice da lista
for i in range(len(processed_lines)):
    # Criar lista de palavras tokenizadas para cada linha
    word_tokens = word_tokenize(processed_lines[i])
    # Inicializar uma lista para receber as palavras
    # filtradas
    filtered_sentence = []
    # Para cada palavra da lista de palavras tokenizadas:
    for word in word_tokens:
        # Se a palavra não está na lista de stop words
        if word not in stop_words:
            # Cria em stemm para a palavra
            stemmed_word = ps.stem(word)
            # Adiciona na lista de palavras filtradas
            filtered_sentence.append(stemmed_word)
    # Junta as palavras da lista para substituir o
    # antigo texto pelo novo texto processado
    processed_lines[i] = ' '.join(filtered_sentence)
```

Construção do modelo de predição

Agora que todo o texto das mensagens de SMS já foi tratado (pré-processado), este é o momento de criar um vetor que servirá de modelo para a vetorização de cada instância do *dataset* (JURAFSKY; MARTIN, 2019), ou seja, será um modelo para as *features* (características). Na verdade, esse vetor compreende uma lista das palavras mais frequentes, presentes em todo o texto do conjunto de dados. Como exemplo, serão selecionadas as 1.500 palavras mais frequentes entre as mensagens SMS, mas que podem ser ajustáveis para melhorar o desempenho. Nesse ponto, é necessário tomar cuidado com as palavras que farão parte do vetor modelo, pois o classificador leva em consideração cada palavra que aparece no conjunto de treinamento, até mesmo aquelas que aparecem uma única vez em todo o texto. Lembre-se: quantidade não significa qualidade.



Exemplo

```
# Inicializar uma lista de palavras
all_words = []

# Para cada linha da lista de linhas que foram processadas:
for line in processed_lines:
    # Criar lista de palavras tokenizadas para cada linha
    word_tokens = word_tokenize(line)
    # Para cada palavra da lista de palavras:
    for word in word_tokens:
        # Adiciona a palavra no bag onde estão todas as
        # palavras
        all_words.append(word)

# Atribui a contagem de frequência para cada palavra da
# lista
all_words = nltk.FreqDist(all_words)

# Imprimir o número total de palavras existentes na lista
# (opcional)
print(f'Total de palavras: {len(all_words)}')
```

```
# Imprimir as 10 palavras mais comuns existente na lista
(opcional)
print(f'Palavras mais comuns: {all _ words.most _ common(10)}')

# Usar as 1500 palavras mais comuns como modelo de features
# O método keys() da classe 'FreqDist' retorna somente
# os termos, que são as chaves da estrutura 'dict _ keys',
# por isso, foi usado list() para converter 'dict _ keys'
# para o tipo list
word _ features = list(all _ words.keys())[0:1500]

# Imprimir as 10 primeiras features (opcional)
print(f'Lista de características: {word _ features[0:10]}')
```

Com o vetor que servirá de modelo já finalizado, podemos transformar cada mensagem, previamente filtrada, em um vetor de *features*. A estrutura que receberá os dados será uma lista de *features* composta por tuplas com dois valores: nas posições 0 das tuplas, contêm os vetores de *features*, e, nas posições 1, os rótulos. Para isso, pode ser usado o código a seguir, que faz uso das bibliotecas scikit-learn (SCIKIT-LEARN, c2020) e numpy (NUMPY, c2020).



Exemplo

```
# Biblioteca numérica Python
import numpy as np

# Biblioteca science kit learning para aprendizagem de
máquina
import sklearn

# Biblioteca para converter valor categórico em discreto
from sklearn.preprocessing import LabelEncoder

# Converter rótulos de classes para valores binários, 0
= ham e 1 = spam
encoder = LabelEncoder()
bin _ labels = encoder.fit _ transform(classes)
```

```
# Adicionar em uma lista de tuplas a mensagem e o rótulo
da classe, de acordo com suas posições. id 0 com id 0, 1
com 1 ...
messages = list(zip(processed_lines, bin_labels))

# definir um valor de seed para reprodutibilidade
seed = 1
# Configura o valor para seed
np.random.seed = seed
# Embaralhar as mensagens para que a ordem não interfira
no treinamento
np.random.shuffle(messages)

# Inicializar uma lista geral para os vetores de features
e os respectivos rótulos
featuresets = []

# Para cada par texto e rótulo da lista de mensagens:
for text, label in messages:
    # Criar lista de palavras tokenizadas para cada linha
    word_tokens = word_tokenize(text)
    # Inicializar um set (conjunto) para as features
    features = {}
    # Para cada palavra existente na lista de tokens modelo
    for word in word_tokens:
        # Se a palavra existir no texto, atribui True,
        # se não, False
        features[word] = (word in word_tokens)
    # Adiciona na lista geral como tupla de (vetor de
    # features e rótulos binarizados)
    featuresets.append((features, label))
```

2 Análise do classificador Naïve Bayes

O classificador Naïve Bayes é um algoritmo de aprendizagem de máquina supervisionado que pertence a uma família de algoritmos probabilísticos baseados na teoria das probabilidades e no teorema de Bayes, capaz de prever o conteúdo de um texto atribuindo-lhe um rótulo (nome de classe). Esse algoritmo faz suposições de maneira simplista sobre como os recursos interagem, por isso o nome Naïve. Assim, ele calcula a probabilidade de cada

uma das classes para um mesmo documento e, em seguida, produz como saída o rótulo da classe que foi marcado com a probabilidade mais alta, a partir do emprego do teorema de Bayes, que descreve a probabilidade de determinado recurso, com base no conhecimento *a priori* (prévio) de condições que podem estar a ele relacionadas (JURAFSKY; MARTIN, 2019).

Para exemplificar, considere um conjunto de seis mensagens de SMS, três classificadas como SPAM e duas como NÃO-SPAM para o treinamento do classificador, além de uma frase de teste para classificação (Quadro 1).

Quadro 1. Tabela de mensagens rotuladas e não rotuladas

	#	Palavras	Classe
Treino	0	Compre Viagra sem juros	SPAM
	1	passar mercado abastecer geladeira	NÃO-SPAM
	2	Viagra barato medicamento	SPAM
	3	Medicamento estimulador preço baixo	SPAM
	4	deixei prato feito geladeira	NÃO-SPAM
Teste	5	Medicamento emagrecedor sem frete	?

Agora, devemos rotular a mensagem “Medicamento emagrecedor sem frete” com qual classe? Como o Naïve Bayes é um classificador probabilístico, pode-se calcular a probabilidade de que a mensagem seja *spam* e *não_spam*.

Teorema de Bayes:

$$P(c|d) = \frac{P(d|c) \times P(c)}{P(d)}$$

- Exemplo de **c** (classe/hipótese): *spam/não_spam*.
- Exemplo de **d** (documento/dados): uma mensagem de SMS.
- **P(c)**: probabilidade *a priori* da hipótese **c** (quanto se pode confiar na hipótese sem ver os dados).
- **P(d|c)**: probabilidade de **d** considerando **c**.
- **P(d)**: probabilidade *a priori* sobre as amostras dos dados do treino **d**.
- **P(c|d)**: Probabilidade de **c** considerando **d**.

$$P(\text{spam}|\text{mensagem}) = \frac{P(\text{mensagem}|\text{spam}) \times P(\text{spam})}{P(\text{mensagem})}$$

Já que, para o classificador em questão, estamos tentando descobrir qual rótulo tem uma probabilidade maior, é possível descartar o divisor, o mesmo para as duas classes, e comparar da seguinte forma:

$$P(\text{mensagem}|\text{spam}) \times P(\text{spam}) \quad \text{com} \quad P(\text{mensagem}|\text{n\~ao_spam}) \times P(\text{n\~ao_spam})$$

Assim, é mais simples calcular essas probabilidades apenas contando quantas vezes a mensagem “Medicamento emagrecedor sem frete” aparece na classe `spam`, e, depois, dividindo pelo total. No entanto, aqui existe um problema, pois a mensagem aparece zero vezes na lista das mensagens de treinamento. Para resolvê-lo, pode-se assumir que cada palavra existente na mensagem é independente das outras:

$$P(\text{medicamento}|\text{spam}) \times P(\text{emagrecedor}|\text{spam}) \times P(\text{sem}|\text{spam}) \times P(\text{frete}|\text{spam})$$

A partir disso, podemos calcular cada probabilidade e verificar qual delas é maior. Calcular uma probabilidade consiste em apenas contar nos dados de treinamento. Primeiro, deve-se calcular a probabilidade *a priori* de cada rótulo, ou seja, para determinada mensagem nos dados de treinamento, a probabilidade de que seja `spam` $P(\text{spam})$ é de 3 em 5, e, para não `spam` $P(\text{n\~ao_spam})$, de 2 em 5. Assim, para calcular $P(\text{medicamento}|\text{spam})$, basta contar quantas vezes a palavra “medicamento” aparece na classe `spam`, ou seja, 2 em 11 palavras da classe `spam`. A próxima probabilidade para calcular seria para $P(\text{emagrecedor}|\text{spam})$, que aparece 0 vez em 11 palavras. Mais uma vez, existe um problema a resolver aqui, pois, na multiplicação das probabilidades, se um dos fatores tem valor zero, o resultado será sempre zero. Uma forma de contornar isso é usando uma técnica chamada *Laplace smoothing* ou *Additive smoothing* (JURAFSKY; MARTIN, 2019), que se trata de adicionar 1 em toda contagem; assim, nunca haverá zero. Por questões de balanceamento, será adicionado 1 para cada uma das 16 palavras distintas do conjunto. Veja as palavras no conjunto e quantas vezes aparecem: compre (1), Viagra (2), sem (1), juro (1), passar (1), mercado (1), abastecer (1), geladeira (2), barato (1), medicamento (2), estimulador (1), preço(1), baixo (1), deixei (1), prato (1), feito (1).

A partir de agora, podemos calcular a probabilidade de cada termo aparecer no conjunto de treinamento fazendo a adição do *smoothing*. O resultado para $P(\text{medicamento}|\text{spam})$ seria $2+1/11+16$, ou seja, medicamento aparece 2 (+ 1) em 11 (+16) palavras da classe spam. Aplicando o mesmo cálculo para cada termo da frase “medicamento emagrecedor sem frete”, temos as informações apresentadas no Quadro 2.

Quadro 2. Tabela de cálculos das probabilidades de uma palavra ocorrer no conjunto rotulado

Palavra	$P(\text{palavra} \text{spam})$		$P(\text{palavra} \text{não-spam})$	
medica- mento	$(2 + 1) / (11 + 16)$	= 0,1111111111111111	$(0 + 1) / (8 + 16)$	= 0,041666666 666667
emagrecedor	$(0 + 1) / (11 + 16)$	= 0,037037037037037	$(0 + 1) / (8 + 16)$	= 0,0416666666 66667
sem	$(0 + 1) / (11 + 16)$	= 0,037037037037037	$(0 + 1) / (8 + 16)$	= 0,04166666666 6667
frete	$(0 + 1) / (11 + 16)$	= 0,037037037037037	$(0 + 1) / (8 + 16)$	= 0,04166666666 6667
Resultado		0,00000564502		0,00000301 408

Aplicando para a equação de spam, tem-se:

$$(P(\text{medicamento}|\text{spam}) \times P(\text{emagrecedor}|\text{spam}) \times P(\text{sem}|\text{spam}) \times P(\text{frete}|\text{spam})) = 0,00000564502$$

E, para a equação de `não_spam`,:

$$\frac{P(\text{medicamento}|\text{não_spam}) \times P(\text{emagrecedor}|\text{não_spam}) \times P(\text{sem}|\text{não_spam}) \times P(\text{frete}|\text{não_spam})}{P(\text{sem}|\text{não_spam}) \times P(\text{frete}|\text{não_spam})} = 0,00000301408$$

De acordo com os resultados obtidos, após submeter a mensagem “medicamento emagrecedor sem frete” aos cálculos de probabilidades, a classe com rótulo “spam” foi a que teve maior probabilidade. Ainda, há as medidas de avaliação dos resultados, como acurácia, precisão, revocação, pontuação F1, além das tabelas de contingência e matriz de confusão, discutidas a seguir.

3 Aplicação do classificador em um estudo prático

Para exemplificar o comportamento do Naïve Bayes, será utilizado um algoritmo chamado Multinomial Naïve Bayes a partir da sua aplicação em um estudo prático. O classificador Multinomial Naïve Bayes é adequado para classificação com recursos discretos (valores numéricos finitos), por exemplo, realizar a contagem de palavras para classificação de texto. A distribuição multinomial normalmente requer contagens de recursos inteiros, conforme realizamos inicialmente neste capítulo.

Treinamento do modelo de predição

Antes de aplicar o classificador sobre o *dataset* de *features*, é necessário dividir a amostra de dados em duas partes, sendo uma parte o **conjunto de dados de treinamento** para treinar o classificador e outra o **conjunto de dados de teste** para os testes de desempenho. Isso possibilitará avaliar a **precisão** e verificar se o modelo se generaliza bem, ou seja, se consegue prever bem os dados que não tinha visto antes; caso contrário, pode ter ocorrido um **overfitting**, que se dá quando o modelo está muito treinado para os dados de treinamento e apenas memorizou os dados de treinamento no lugar de “aprendê-los”. Esse comportamento representaria uma alta precisão de predição nos dados de treinamento, mas baixa nos dados de teste. Vale ressaltar ainda que existe um ou outro conjunto de dados conhecido como **conjunto de dados de desenvolvimento**, denominado dessa maneira, pois geralmente é utilizado enquanto se desenvolve o classificador com diferentes configurações ou variações na representação do recurso.

Para dividir a amostra de dados, usaremos o método `train_test_split`, da biblioteca `model_selection` do *scikit-learn*, que serve para dividir um único conjunto de dados para dois propósitos diferentes: treinamento e teste. O subconjunto de treinamento é utilizado para construir o modelo de predição, e o de teste, por sua vez, para usar o modelo em dados desconhecidos, a fim de avaliar o desempenho do modelo. Por padrão, `train_test_split` faz partições aleatórias para os dois subconjuntos (treinamento e teste). No entanto, é possível especificar um estado aleatório para a operação ou um estado fixo para que consiga ser reproduzível. Os parâmetros básicos de `train_test_split` são:

- `*arrays`: conjunto de dados, cujos tipos permitidos são listas, matrizes *numpy*, matrizes *scipy-sparse* ou *pandas data frame*;
- `train_size`: define o tamanho do conjunto de dados de treinamento. Existem três opções: nenhuma, o padrão; *Int*, que requer o número exato de amostras; e *float*, que varia de 0,1 a 1,0;
- `test_size`: especifica o tamanho do conjunto de dados de teste. A configuração-padrão se adequa ao tamanho do treinamento. É definido como 0,25 se o tamanho do treinamento estiver definido como padrão;
- `random_state`: o modo-padrão executa uma divisão aleatória usando *numpy.random*. Como alternativa, é possível adicionar um número inteiro usando um número exato.



Exemplo

```
# Biblioteca para dividir o dataset
from sklearn.model_selection import train_test_split

# dividir os dados em dois conjuntos; 75% treinamento e 25% teste
training, testing = train_test_split(
    featuresets, # dataset com as características e rótulos
    train_size=None,
    test_size = 0.25, # Tamanho do conjunto de teste
    # garante que as divisões geradas sejam reproduzíveis
    # a variável seed já foi configurada em outro bloco
    de código
```

```
    random_state=seed
)

# Imprimir número de instâncias no conjunto de treinamento (opcional)
print(len(training))

# Imprimir número de instâncias no conjunto de teste (opcional)
print(len(testing))

test_features, test_labels = zip(*testing)
train_features, train_labels = zip(*training)
```

Como último passo no exemplo do código apresentado, foram separadas as características (*features*) dos rótulos (*labels*) dos conjuntos de treinamento e teste. Isso se deve ao fato de que foi passado como primeiro parâmetro para `train_test_split` o *dataset* completo, ou seja, características e rótulos na mesma estrutura de dados por meio da variável *featuresets*. É possível fazer com que `train_test_split` retorne os rótulos separados das características, para o qual os rótulos também devem ser enviados ao método separados, como mostrado no exemplo a seguir.



Exemplo

```
# Bloco de código meramente informativo (NÃO USAR EM CON-
JUNTO COM OS OUTROS BLOCOS DE CÓDIGO)
features_train, features_test, labels_train, labels_
test = train_test_split(
    features, # Conjunto de características
    labels, # Conjunto de rótulos
    train_size=None,
    test_size=0.25,
    random_state=1
)
```

Com os dados separados para treinamento e teste, agora podemos realizar a classificação propriamente dita. Para isso, será utilizada a classe `MultinomialNB` do pacote `naive_bayes` do `scikit-learn`. São três os parâmetros da classe `MultinomialNB`, que configuram o comportamento do algoritmo Naïve Bayes:

- `alpha`: (*float*, opcional, `default=0`). Aditivo (Laplace/Lidstone) parâmetro de suavização (0 para não suavização);
- `fit_prior`: (*boolean*). Se deve aprender probabilidades das classes anteriores ou não. Se `False`, uma classe anterior será usada;
- `class_fit`: (tipo *array* de tamanho *n_classes*). Probabilidades das classes anteriores. Se especificado, os “priors” não são ajustados de acordo com os dados.

No entanto, há uma alteração da estrutura de dados a ser realizada sobre os dados de treinamento e teste, já que os dados de entrada para a classe `MultinomialNB` precisam estar no formato de matriz, ou seja, lista de listas. Porém, os dados representados pelas variáveis `train_features` e `train_test` estão no formato da estrutura de dados do tipo dicionário. Para realizar essa conversão, será utilizada a classe `DictVectorizer` da biblioteca `feature_extraction`. Após a conversão de dicionário para matriz, trata-se do momento de enviar os dados para treinamento e teste do modelo de predição.



Exemplo

```
# Biblioteca do classificador multinomial Naive Bayes
from sklearn.naive_bayes import MultinomialNB

# Biblioteca para vetorizar dicionários
from sklearn.feature_extraction import DictVectorizer

# Instanciar
vect = DictVectorizer(sparse=False)
```

```
# Vetorizar os dados de treinamento e teste
train_features = vect.fit_transform(train_features)
test_features = vect.fit_transform(test_features)

# Instanciar o modelo Naive Bayes
model = MultinomialNB(alpha=1.0, fit_prior=True,
class_prior=None)

# Treinar o modelo com os dados de treinamento
model.fit(train_features, train_labels)

# Testar o modelo com os dados de teste
predicted = model.predict(test_features)
```

Avaliação do modelo de predição

A partir desse momento, já conseguimos submeter os resultados da predição sobre o conjunto de teste às métricas de avaliação de desempenho. No entanto, antes de ser apresentado um exemplo prático em código para exibição da avaliação, devemos compreender alguns conceitos das métricas que serão utilizadas para a avaliação do modelo. Para isso, pode ser tomada como base uma tabela de contingência (JURAFSKY; MARTIN, 2019) de tamanho 2×2 , já que há apenas duas classes no *dataset* utilizado, ou seja, spam e ham (não-spam) (Quadro 3).

Quadro 3. Tabela de contingência 2×2

		Valor predito	
		classe spam	classe não-spam
Valor real	spam	VP: verdadeiro-positivo	FP: falso-positivo
	ham	FN: falso-negativo	VN: verdadeiro-negativo

O Quadro 3 é chamado de **matriz de confusão** e demonstra os erros e acertos do modelo de predição (JURAFSKY; MARTIN, 2019) por meio da comparação entre os valores reais e os que foram preditos.

- VP (verdadeiro-positivo): valor predito: spam. Classe correta: spam.
- FN (falso-negativo): valor predito: não_spam. Classe correta: spam.
- FP (falso-positivo): valor predito: spam. Classe correta: ham.
- VN (verdadeiro-negativo): valor predito: não_spam. Classe correta: ham.

A partir da contagem de erros e acertos e a obtenção da matriz de confusão, é possível avaliar o desempenho do modelo de predição (JURAFSKY; MARTIN, 2019) fazendo uso das métricas descritas a seguir.

- Acurácia: demonstra o quanto o modelo foi assertivo para todas as classes. Sua fórmula é: $(VP + VN) / (VP + VN + FP + FN)$.
- Precisão: indica o quanto o modelo foi assertivo apenas para uma das classes, cuja fórmula é: $VP / (VP + FP)$.
- Revocação: demonstra a frequência dos exemplos que foram encontrados de determinada classe. Sua fórmula é: $VP / (VP + FN)$.
- F1-score: aponta a qualidade do modelo de predição pela média harmônica entre precisão e revocação. Quanto **maior** for o valor, melhor é o modelo de predição. Sua fórmula é: $(2 * precisão * revocação) / (precisão + revocação)$.

Com os conceitos a fundamentados, o próximo passo consistirá em gerar a matriz de confusão dos valores preditos pelo modelo. Para isso, será utilizada a classe `confusion_matrix` do pacote `metrics` do `scikit-learn`.



Exemplo

```
# Biblioteca para gerar matriz de confusão
from sklearn.metrics import confusion_matrix

# Gera matriz de confusão a partir dos resultados preditos
confus_matrix = confusion_matrix(test_labels, predicted)

# Cria um dataframe adicionando nomes de linhas e colunas
pd.DataFrame(
    confus_matrix,
    index = [['Real', 'Real'], ['spam', 'ham']],
    columns = [['Predito', 'Predito'], ['spam', 'ham']]
)
```

Além da matriz de confusão, podemos gerar um relatório contendo a avaliação de desempenho do modelo de predição com os valores aferidos para cada uma das métricas tratadas neste tópico. Para isso, pode ser utilizada a biblioteca `classification_report` do pacote `metrics` do `scikit-learn`.



Exemplo

```
# Biblioteca para gerar relatório de avaliação
from sklearn.metrics import classification_report

# Gerar relatório de avaliação de performance do modelo
report = classification_report(
    test_labels,
    predicted,
    target_names=['spam', 'ham'],
    output_dict=True
)

# Cria um dataframe do relatório
pd.DataFrame(report).transpose()
```

Ajuste dos hiperparâmetros

Geralmente, os algoritmos classificadores apresentam vários parâmetros que podem ser ajustados para obtermos uma melhora nos índices de acertos do modelo de predição (JURAFSKY; MARTIN, 2019). Por exemplo, o classificador Multinomial Naïve Bayes inclui três parâmetros para ajustes, sendo um de suavização e dois de configurações *a priori*. No entanto, descobrir quais são os melhores valores para esses parâmetros torna-se uma tarefa quase impossível de realizar manualmente. Por exemplo, podemos imaginar hipoteticamente um classificador com dois parâmetros de ajustes, sendo o primeiro um que aceita valores inteiros de 1 a 100, e o segundo um que aceita valores booleanos Verdadeiro ou Falso. Com base nessas informações, é possível ter $100 \times 2 = 200$ ajustes diferentes de parâmetros (hiperparâmetros), o que torna o ajuste manual quase impossível, dependendo da quantidade de parâmetros.

Uma das soluções para mitigar o problema do ajuste dos hiperparâmetros (Tune Hyperparameters) consiste em trabalhar com uma matriz de configuração de todos os parâmetros e testar essas combinações de forma exaustiva ou randômica. A técnica Grid Search testa de forma exaustiva todas as combinações possíveis, enquanto a Random Search testa combinações aleatórias dos valores dos parâmetros. Quando o conjunto de dados é relativamente pequeno, mas há muitos ajustes de parâmetros, o método Grid Search produz resultados mais precisos. No entanto, com grandes conjuntos de dados e muitas dimensões para ajuste, o tempo de computação será muito alto, caso para o qual se recomenda usar a técnica Random Search, pois a quantidade de testes de combinações de parâmetros é manualmente ajustável.

Para o exemplo de código de ajustes dos hiperparâmetros, será utilizada a biblioteca GridSearchCV do pacote `model_selection`, que otimiza os ajustes por validação cruzada.



Exemplo

```
# Biblioteca para buscar melhores parâmetros
from sklearn.model_selection import GridSearchCV

parameters = {
    # linspace do numpy gera uma lista entre
    # intervalos (start, stop, lenght)
    'alpha': np.linspace(0.1, 1.0, 10),
    # Ligado ou desligado
    'fit_prior': [True, False],
    # Nenhum ou [probabilidade para cada classe]
    'class_prior': [None, [0.25, 0.5]]
}

# Instanciar o GS com o estimador (model), o grid
# de parâmetros a serem testados, a estratégia de
# busca por validação cruzada com 5 folds e a quantidade
# de processadores em paralelo para a busca (-1 usa todos)
gs = GridSearchCV(model, parameters, cv=5, n_jobs=-1)

# Treinar o modelo N vezes com todas as combinações de
# parâmetros. Esse processo pode demorar, dependendo
# da quantidade de parâmetros.
gs.fit(train_features, train_labels)

# Imprimir melhor score e melhores parâmetros
print("Melhor score: ", gs.best_score_)
print("Melhores parâmetros: ", gs.best_params_)
```

Ao final da execução do bloco do código apresentado, serão impressos o maior valor alcançado do *score* e os valores dos parâmetros utilizados para obtê-lo. Com base no que foi discutido até agora, podemos elencar alguns argumentos favoráveis ou contrários ao classificador Naïve Bayes.

Apesar de sua ingenuidade, o algoritmo faz um bom trabalho na predição da classe correta, além de funcionar muito bem quando aplicado a situações do mundo real, como classificação de documentos, filtragem de *spam*, etc. Além disso, na maioria das vezes, esse tipo de classificador é rápido na execução, já que consegue ter um bom desempenho com poucos dados. Também, é relativamente simples de configurar para aplicações rotineiras.

Em contrapartida, a suposição de independência entre os recursos retira um pouco a capacidade de explorar as interações que ocorrem entre os dados. No mundo real, é quase impossível um conjunto de dados ter atributos independentes, o que não se tem mostrado um problema para tarefas de classificação de texto. Não menos importante, há a possibilidade de existir uma variável no conjunto de dados de teste que não foi observada no conjunto de dados de treinamento. Assim, o modelo atribui uma probabilidade 0 (zero) e não consegue fazer uma previsão precisa.

Este capítulo abordou a classificação supervisionada de texto com Python, a partir da apresentação dos processos de manipulação de dados que podem ser utilizados em classificadores de texto. Isso fornece a você a possibilidade de compreender passo a passo a como tratar os dados, com o uso da linguagem de programação Python, antes de submetê-los a um classificador de texto. Ainda neste capítulo, foi apresentado o classificador Naïve Bayes, de fácil implementação e uso para a detecção, por exemplo, de *spam* em mensagens de texto. Por fim, o classificador Naïve Bayes foi utilizado em um estudo prático de detecção de *spam* em mensagem de SMS, além de termos demonstrado como aferir o desempenho alcançado a partir dele. Posto isso, convidamos o leitor a praticar os assuntos apresentados no capítulo e tentar obter os melhores resultados na classificação de texto com a aplicação das diferentes técnicas na preparação dos dados e no ajuste de parâmetros.



Referências

DUA, D.; GRAFF, C. *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science, 2019. Disponível em: <https://archive.ics.uci.edu/ml/index.php>. Acesso em: 14 maio 2020.

GOOGLE COLABORATORY. Mountain View, CA: Google, c2020. Disponível em: <https://colab.research.google.com>. Acesso em: 14 maio 2020.

JURAFSKY, D. S.; MARTIN, H. *Speech and language processing: an introduction to natural language processing, computational linguistics, and speech recognition*. 3. ed. New Jersey: Prentice Hall, 2019.

NATURAL LANGUAGE TOOLKIT. NLTK 3.5 documentation. 2020. Disponível em: <http://www.nltk.org>. Acesso em: 14 maio 2020.

NUMPY. c2020. Disponível em: <https://numpy.org>. Acesso em: 14 maio 2020.

PANDAS. Python Data Analysis Library. Texas: Zenodo, feb. 2020. Disponível em: <https://pandas.pydata.org>. Acesso em: 14 maio 2020.

PYTHON. Wilmington: Python Software Foundation, c2020. Disponível em: <https://www.python.org/>. Acesso em: 14 maio 2020. SCIKIT-LEARN: Machine learning in Python. c2020. Disponível em: <https://scikit-learn.org>. Acesso em: 14 maio 2020.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS